

Universidad Nacional Autónoma de México



Facultad de Ingeniería

División de Ingeniería Eléctrica



Laboratorio Computación Gráfica e Interacción
Humano-Computadora (6590)

Práctica 5: Optimización y Carga de Modelos

Nombre: Calderón Gutiérrez Victor Emiliano

No. Cuenta: 423097331

Grupo (laboratorio): 03

Grupo (teoría): 02

Semestre 2026-1

Fecha de entrega: 28/09/2025

Calificación: _____

1.Ejercicios

Para esta práctica se buscó comenzar la implementación para comenzar a usar modelos creados por otras personas para después modificarlos, optimizarlos y cambiarlos para que sean más útiles y se pueda implementar cosas que se han visto previamente.

Lo primero que se hizo fue la creación de las variables para añadir los modelos:

```
// Modelos para el coche
Model coche_cuerpo;
Model coche_rueda_izquierda_1;
Model coche_rueda_izquierda_2;
Model coche_rueda_derecha_1;
Model coche_rueda_derecha_2;
Model coche_cofre;
```

Posteriormente se comenzó con la carga de los modelos en esas variables:

```
// Carga de modelos para el coche
coche_cuerpo = Model();
coche_cuerpo.LoadModel("Models/coche_cuerpo.obj");
coche_rueda_izquierda_1 = Model();
coche_rueda_izquierda_1.LoadModel("Models/coche_rueda_izquierda_1.obj");
coche_rueda_izquierda_2 = Model();
coche_rueda_izquierda_2.LoadModel("Models/coche_rueda_izquierda_2.obj");
coche_rueda_derecha_1 = Model();
coche_rueda_derecha_1.LoadModel("Models/coche_rueda_derecha_1.obj");
coche_rueda_derecha_2 = Model();
coche_rueda_derecha_2.LoadModel("Models/coche_rueda_derecha_2.obj");
coche_cofre = Model();
coche_cofre.LoadModel("Models/coche_cofre.obj");
```

Para poder terminar generando todos los renderizados de objeto:

```

// Coche
model = glm::mat4(1.0);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -1.5f, -10.5f));
// Movimiento adelante y atrás
model = glm::translate(model, glm::vec3(-mainWindow.getMovimientoAdelante(), 0.0f, 0.0f));
model = glm::translate(model, glm::vec3(mainWindow.getMovimientoAtras(), 0.0f, 0.0f));
modelcuerpo = model;
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
coche_cuerpo.RenderModel();

// Color para diferenciar modificaciones
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));

// Rueda izquierda 1
model = modelcuerpo;
model = glm::translate(model, glm::vec3(-8.85f, 3.25f, 5.75f));
// Rotación rueda adelante
model = glm::rotate(model, glm::radians(mainWindow.getRuedaAdelante()), glm::vec3(0.0f, 0.0f, -1.0f));
// Rotación rueda atrás
model = glm::rotate(model, glm::radians(mainWindow.getRuedaAtras()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
coche_rueda_izquierda_1.RenderModel();

```

```

// Rueda izquierda 2
model = modelcuerpo;
model = glm::translate(model, glm::vec3(18.0f, 3.25f, 5.75f));
// Rotación rueda adelante
model = glm::rotate(model, glm::radians(mainWindow.getRuedaAdelante()), glm::vec3(0.0f, 0.0f, -1.0f));
// Rotación rueda atrás
model = glm::rotate(model, glm::radians(mainWindow.getRuedaAtras()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
coche_rueda_izquierda_2.RenderModel();

// Rueda derecha 1
model = modelcuerpo;
model = glm::translate(model, glm::vec3(-8.85f, 3.25f, -5.75f));
// Rotación rueda adelante
model = glm::rotate(model, glm::radians(mainWindow.getRuedaAdelante()), glm::vec3(0.0f, 0.0f, -1.0f));
// Rotación rueda atrás
model = glm::rotate(model, glm::radians(mainWindow.getRuedaAtras()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
coche_rueda_derecha_1.RenderModel();

// Rueda derecha 2
model = modelcuerpo;
model = glm::translate(model, glm::vec3(18.0f, 3.25f, -5.75f));
// Rotación rueda adelante
model = glm::rotate(model, glm::radians(mainWindow.getRuedaAdelante()), glm::vec3(0.0f, 0.0f, -1.0f));
// Rotación rueda atrás
model = glm::rotate(model, glm::radians(mainWindow.getRuedaAtras()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
coche_rueda_derecha_2.RenderModel();

```

```
// Cofre del coche
model = modelcuerpo;
model = glm::translate(model, glm::vec3(15.25f, 5.0f, 0.0f));
// Abrir cofre
model = glm::rotate(model, glm::radians(mainWindow.getAbrirCofre()), glm::vec3(0.0f, 0.0f, -1.0f));
// Cerrar cofre
model = glm::rotate(model, glm::radians(mainWindow.getCerrarCofre()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
coche_cofre.RenderModel();

// Goddard de referencia para el tamaño del coche
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-25.0f, -1.5f, 0.0f));
color = glm::vec3(0.0f, 1.0f, 0.0f); // Color verde para diferenciar
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Goddard.RenderModel();
```

Para las rotaciones de cada parte de los objetos se creó como se hizo en prácticas y ejercicios anteriores, se comenzó creando variables en Window.cpp y Window.h para poder aumentar y disminuir los ángulos. El único diferente fue el del desplazamiento, pero se implementó con algo similar como en las rotaciones, se puso la variable que se modifica dentro del eje de las x con signo distinto para así simular que avanza o retrocede.

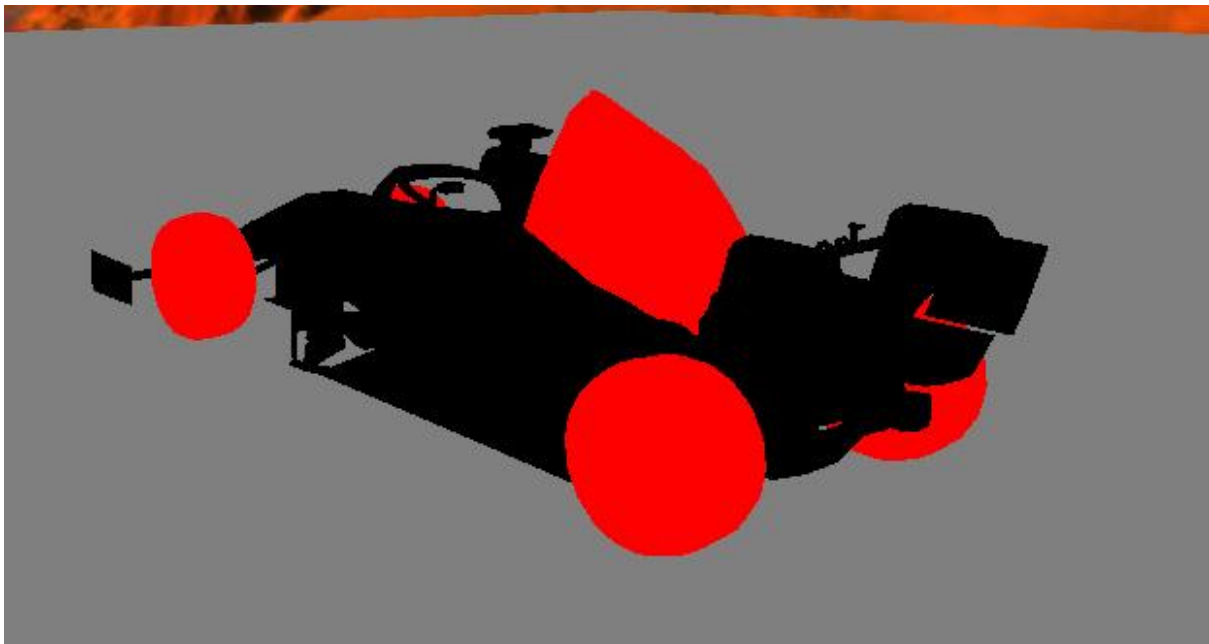
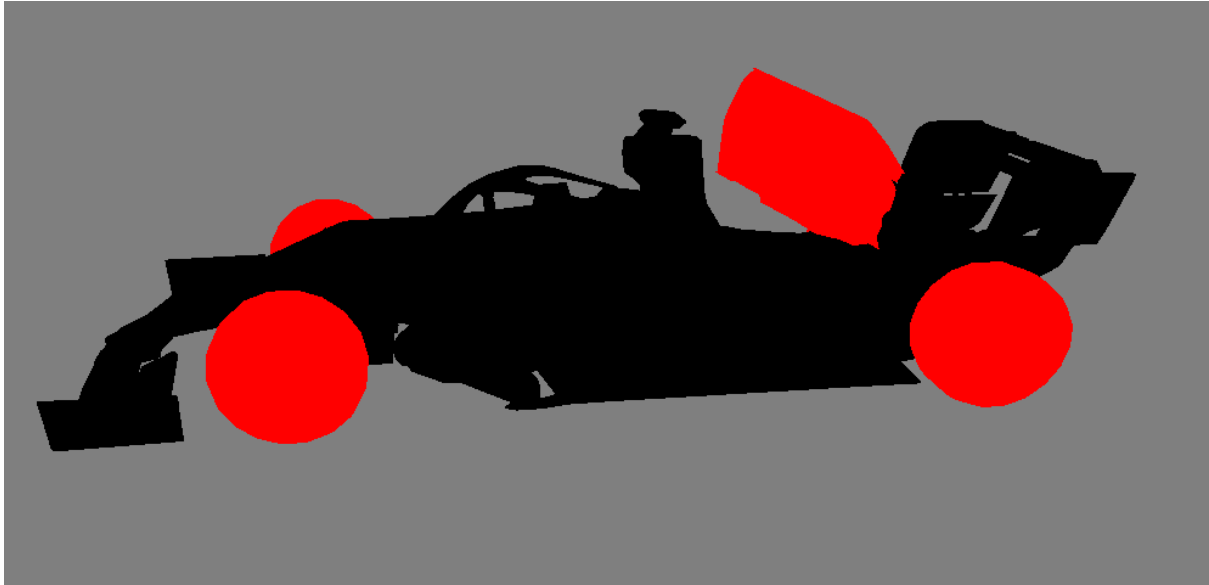
```
GLfloat getrotaz() { return rotaz; }
GLfloat getRuedaAdelante() { return rueda_adelante; }
GLfloat getRuedaAtras() { return rueda_atras; }
GLfloat getMovimientoAdelante() { return movimiento_adelante; }
GLfloat getMovimientoAtras() { return movimiento_atras; }
GLfloat getAbrirCofre() { return cofre_abre; }
GLfloat getCerrarCofre() { return cofre_cierra; }

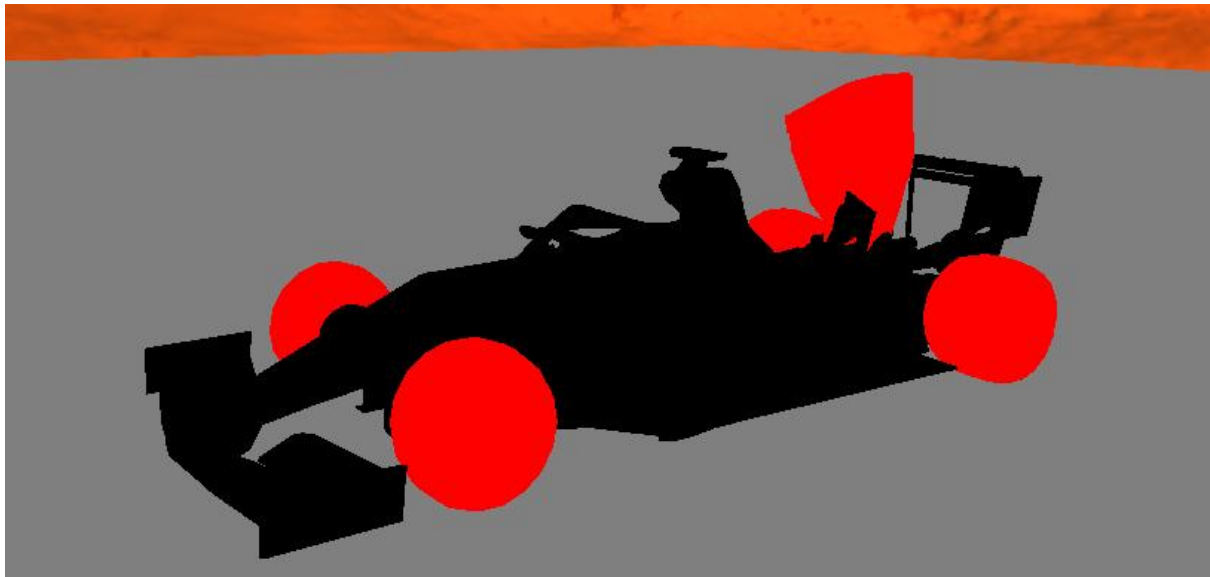
~Window();
private:
GLFWwindow *mainWindow;
GLint width, height;
GLfloat rotay, rotax, rotaz, rueda_adelante, rueda_atras, movimiento_adelante, movimiento_atras, cofre_abre, cofre_cierra;
bool keys[1024];
```

Las ruedas se rotan con F y G, el cofre se abre y cierra con H y J, mientras que el movimiento de todo el coche se hace con K y L.

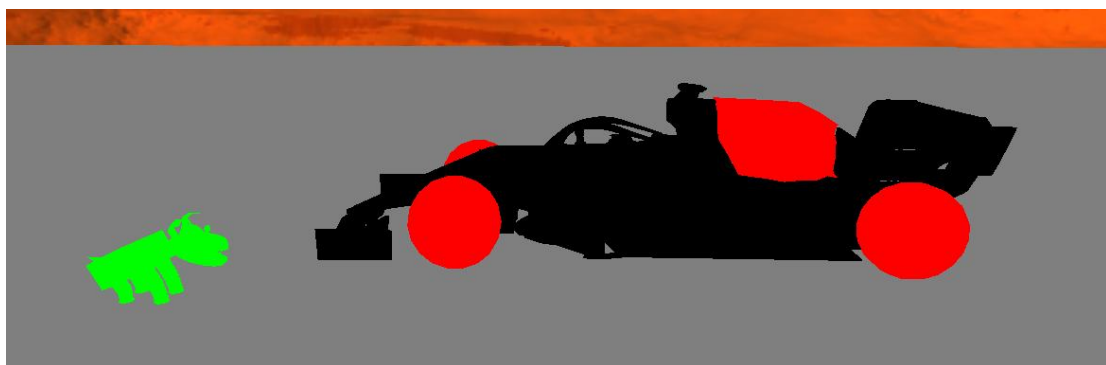
```
if (key == GLFW_KEY_F)
{
    theWindow->rueda_adelante += 10.0;
}
if (key == GLFW_KEY_G)
{
    theWindow->rueda_atras += 10.0;
}
if (key == GLFW_KEY_H)
{
    theWindow->cofre_abre += 10.0;
}
if (key == GLFW_KEY_J)
{
    theWindow->cofre_cierra += 10.0;
}
if (key == GLFW_KEY_K)
{
    theWindow->movimiento_adelante += 1.0;
}
if (key == GLFW_KEY_L)
{
    theWindow->movimiento_atras += 1.0;
}
```

El coche se ve como en las imágenes, además, aunque las rotaciones no se ven del todo en las imágenes se adjuntan un par de imágenes para ver algo de los movimientos, aunque se invita a revisar el código y también los ejecutables para confirmar el movimiento.





Por último, una de las cosas que se pedían era que fuera de una escala adecuada. Lo que decidí para asegurarme que fuera del tamaño correcto fue que le puse un Goddard de referencia, para de esta forma asegurarme de que el coche sea mucho más grande, pero sea algo dentro de lo considerado normal.



2.Problemas

Esta práctica creo que fue un tanto sencilla en comparación con las previas que hemos hecho. Se pudo implementar de manera eficiente todo lo que hemos visto previamente en el curso, por lo que no se complicó la entrega en esta ocasión.

3.Conclusión

En la realización de esta práctica me encontré bastante entretenido, ya que todo lo que habíamos visto previamente se aplicó y tomó total sentido a la hora de comenzar a trabajar con otros modelos. Siento que ayudó bastante a asentar las bases de todo lo aprendido y me ayudó a ir pensando en formas de poder optimizar el código para tratar de evitar código repetitivo, sé que aún se puede simplificar más pero de momento esta práctica me empujó para poder aplicar todo lo visto. Debo admitir, que

esta práctica por lo mismo tuvo una complejidad un poco más baja que en prácticas anteriores, pero eso no demerita que fue una buena forma de seguir aplicando y reforzando conocimientos.

4. Bibliografía

- Sketchfab. (s. f.-e). Red Bull Racing - Download free 3D model by Jan Esch (@NJan). Recuperado 28 de septiembre de 2025, de <https://sketchfab.com/3d-models/red-bull-racing-c2703044775148ba8f1b5bbfbfcd7938>